

6.5830/1 Lecture 6: Query Execution

2/24/2026

Tianyu Li

litianyu@mit.edu

Many thanks to Andy
Pavlo and CMU DB for
sharing their materials

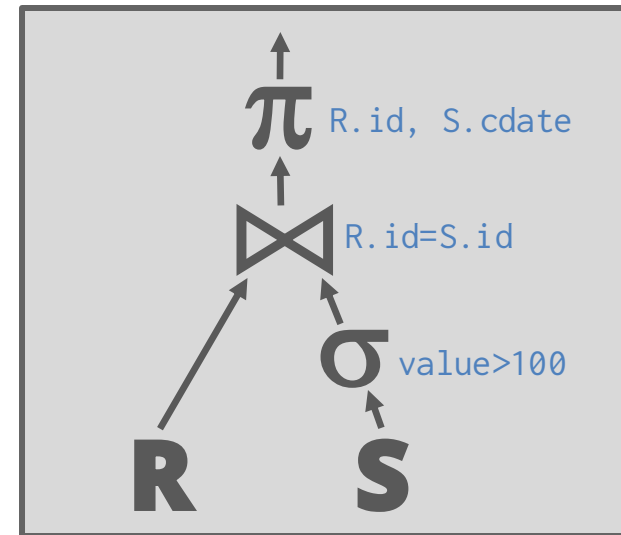
Administrivia

- Lab 1 due Thursday
 - Bootcamp recording available online
 - Come to OH!
- Submit project teams on Thursday
 - If doing a final project
 - Submit on gradescope
- Lab 2 out today
 - Due 3/19
 - Significantly more code than lab 1, so start early and ask questions!
 - Bootcamp info available soon

Recap: Query Plan

- A query plan is a DAG of **operators**. The operators are arranged in a tree.
- Data flows from the leaves of the tree up towards the root.
- The output of the root node is the result of the query.

```
SELECT R.id, S.cdate  
FROM R JOIN S  
ON R.id = S.id  
WHERE S.value > 100
```



Query Processing

- A DBMS's processing model defines how the system executes a query plan and moves data from one operator to the next.
 - Different trade-offs for workloads.
- Different trade-offs for workloads
 - Online Transactions Processing (OLTP): small, short, repetitive, highly concurrent
 - Online Analytical Processing (OLAP): large, complex, often scans entire tables
 - Traditional DBMSs try to do both, but newer systems almost always specialize

Query Processing

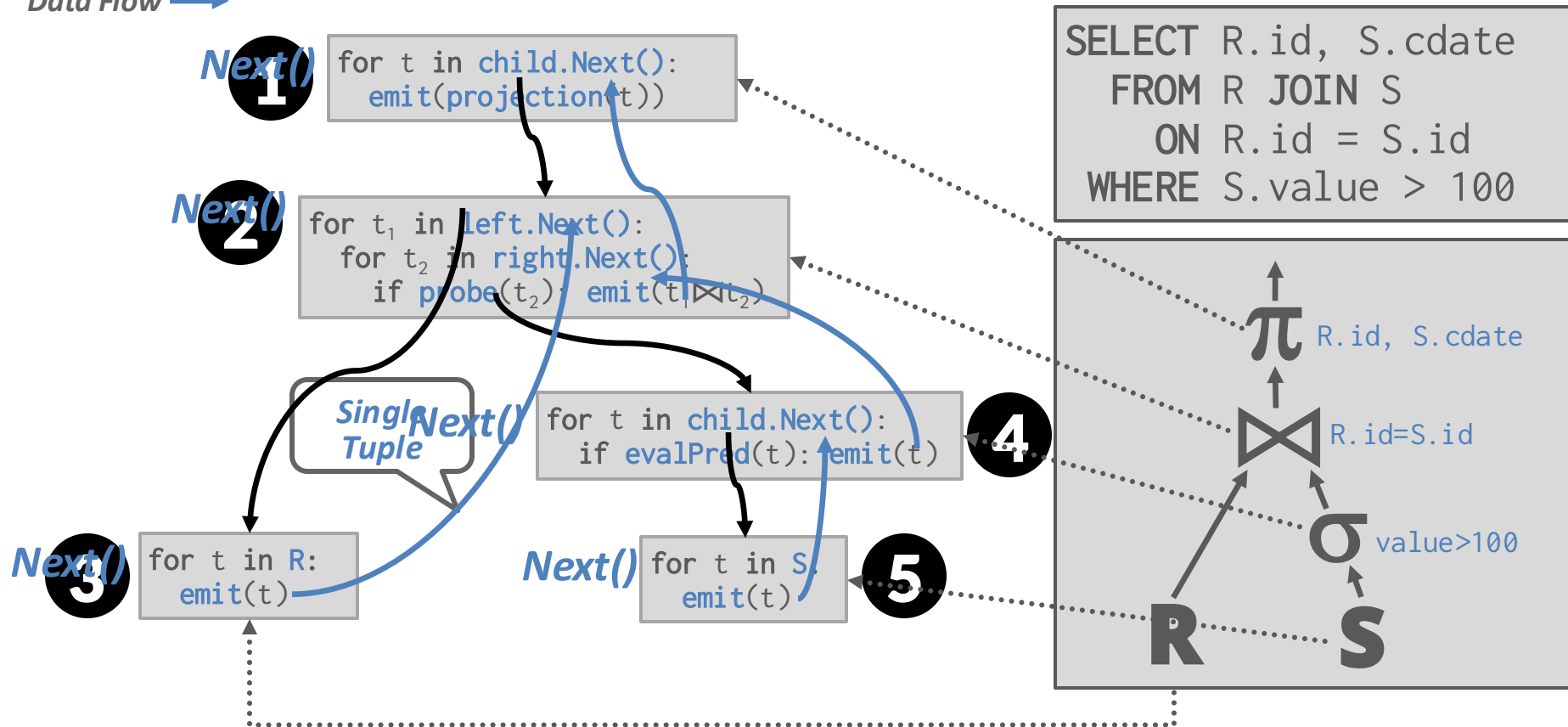
- **Approach #1: Iterator Model** ← *Most Common*
- **Approach #2: Materialization Model** ← *Rare*
- **Approach #3: Vectorized Model** ← *Common*

Iterator Model

- Each query plan node implements a **Next()** function.
 - On each invocation, the operator returns either a single tuple or a EOF marker if there are no more tuples.
 - The operator implements a loop that calls **Next()** on its children to retrieve their tuples and then process them.
- Each operator implementation also has **Init()** and **Close()** functions.
 - Analogous to constructors/destructors, but for operators.
- Also called **Volcano** Model.

Iterator Model

Control Flow $\rightarrow$
Data Flow $\rightarrow$

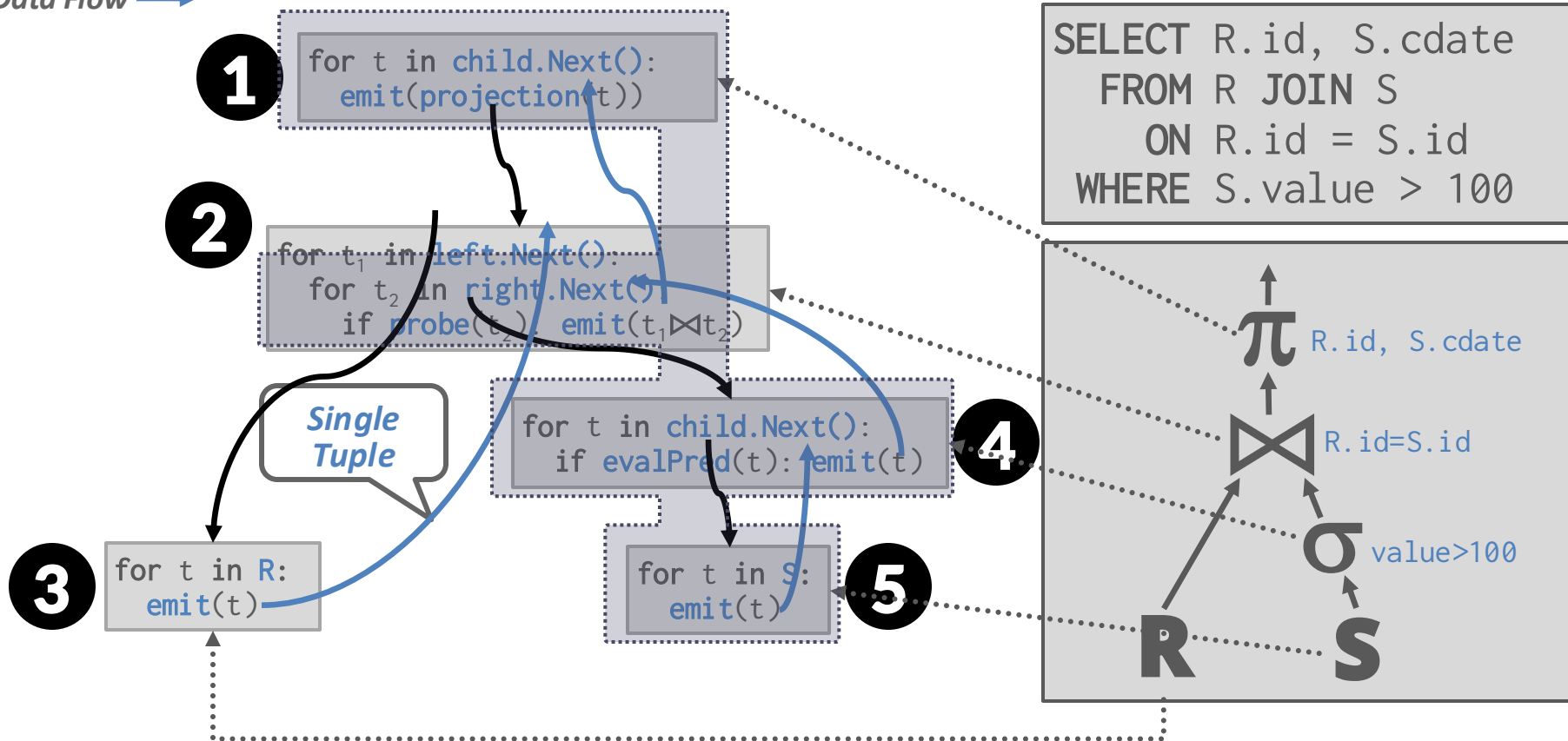


Iterator Model

- The Iterator model is used in almost every DBMS.
 - Easy to implement / debug.
 - Output control works easily with this approach.
- Allows for **pipelining** where the DBMS tries to process each tuple through as many operators as possible before retrieving the next tuple.

Iterator Model

Control Flow $\rightarrow$
Data Flow $\rightarrow$



Iterator Model

- Can we always pipeline?
- No – some operators consume all inputs before emitting
 - E.g., Sorting, Aggregation

Modification Queries

- Operators that modify the database (**INSERT**, **UPDATE**, **DELETE**) are responsible for modifying the target table and its indexes.
- Modification operators are also structured like iterators for simplicity
 - But they have side effects
 - SQL convention: return number of tuples touched

Modification Queries

- **INSERT:**
 - Operator inserts any tuple passed in from child operators.
- **UPDATE/DELETE:**
 - Child operators pass Record IDs for target tuples.
 - Is this sufficient?

Problem

Control Flow 
Data Flow 

```
for t in child.Next():  
    removeFromIndex(idx_salary, t.salary, t)  
    updateTuple(t.salary = t.salary + 100)  
    insertIntoIndex(idx_salary, t.salary, t)
```

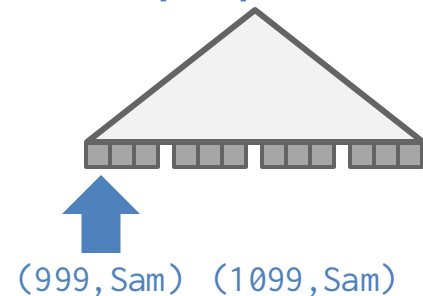
```
for t in Indexpeople:  
    if t.salary < 1100:  
        emit(t)
```

```
CREATE INDEX idx_salary  
ON people (salary);
```

```
UPDATE people  
SET salary = salary + 100  
WHERE salary < 1100
```



Index(people.salary)



Halloween Problem

- Anomaly where a modification operation causes a scan operator to visit the tuple multiple times.
 - Can happen on insert / updates
- First discovered by IBM researchers while working on System R on Halloween day in 1976.
- **Solution:** Track modified record ids per query.
 - Alternatively, exhaust child and store values (i.e., materialize) before invoking modification (GoDB)

Problems with Iterator?

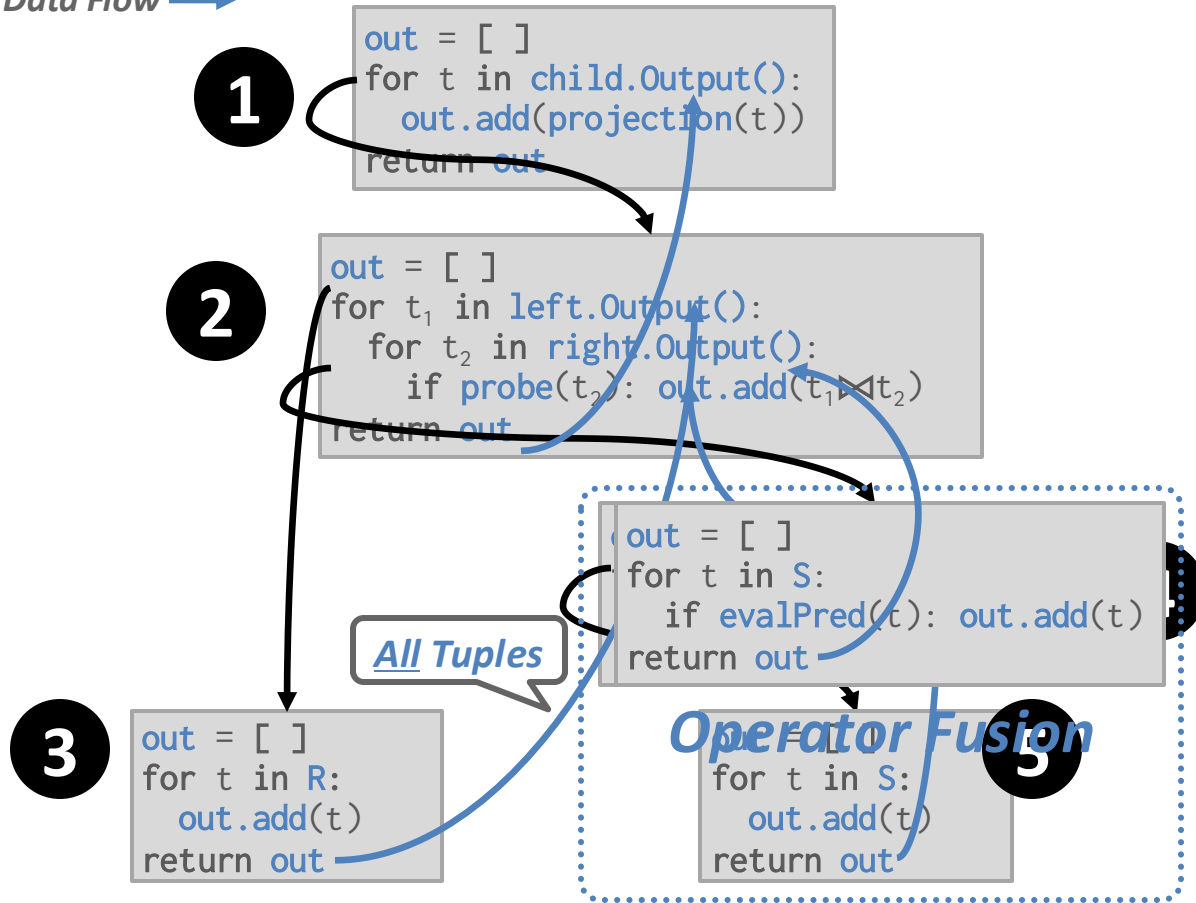
- Every operator involves a virtual function call
 - Many extra instructions to run and many random memory jumps to perform
 - Orders of magnitude slower than writing a for loop for your data
- Why does everyone do it?
 - In many cases, query execution is not the bottleneck
 - But in modern settings, it often is

Materialization Model

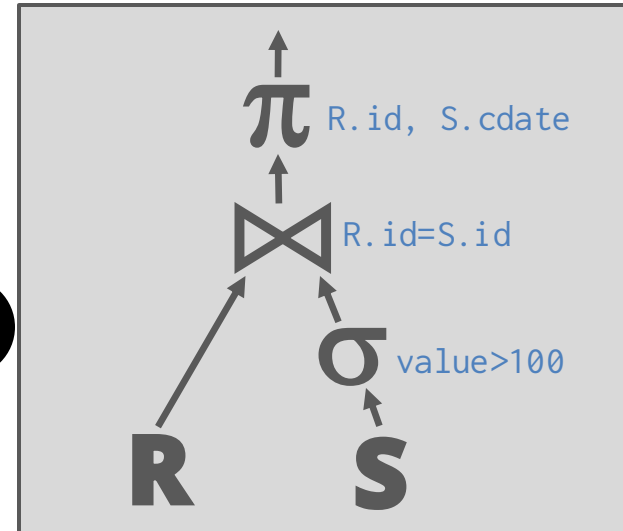
- Each operator processes its input all at once and then emits its output all at once.
 - The operator "materializes" its output as a single result.
 - The DBMS can push down hints (e.g., **LIMIT**) to avoid scanning too many tuples.

Materialization Model

Control Flow →
Data Flow →



```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```



Materialization Model

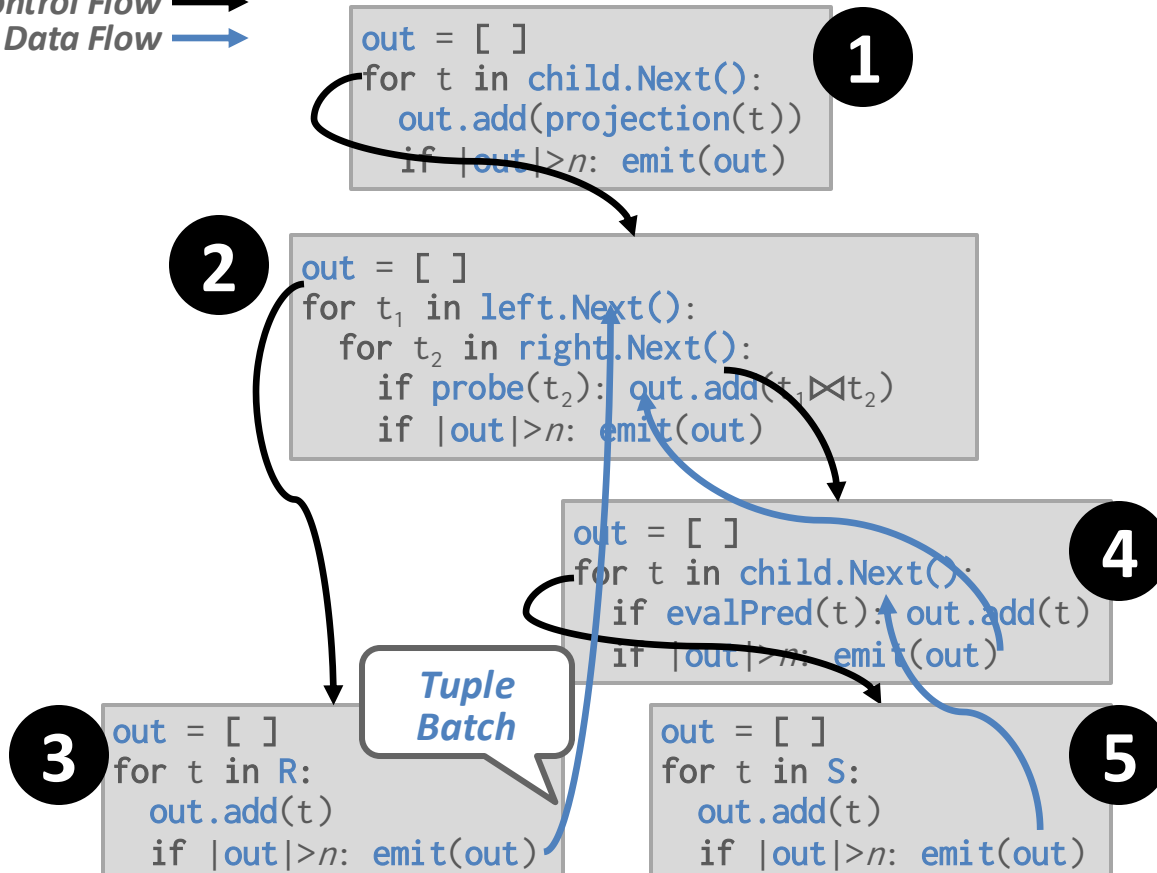
- Better for OLTP workloads because queries only access a small number of tuples at a time.
 - Lower execution / coordination overhead.
- Not ideal for OLAP queries with large intermediate results because DBMS must hold (large) intermediate result sets in memory.

Vectorization Model

- Like the Iterator Model where each operator implements a `Next()` function, but...
- Each operator emits a **batch** of tuples instead of a single tuple.
 - The operator's internal loop processes multiple tuples at a time.
 - The size of the batch can vary based on hardware or query properties.
 - Each batch will contain one or more columns each their own null bitmaps.

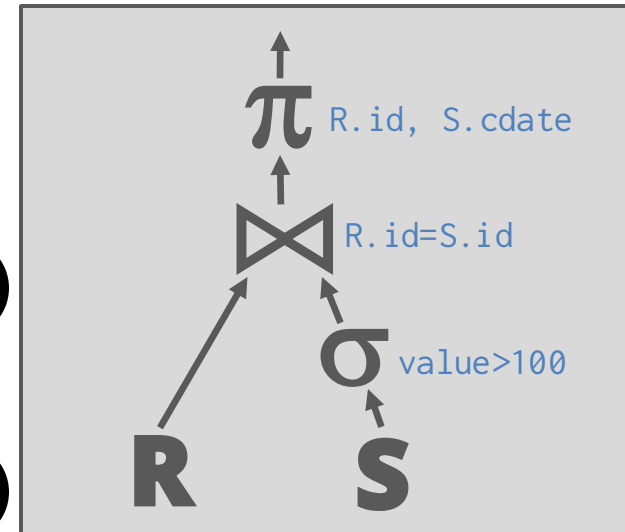
Vectorization Model

Control Flow $\rightarrow$
Data Flow $\rightarrow$



```

SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
    
```



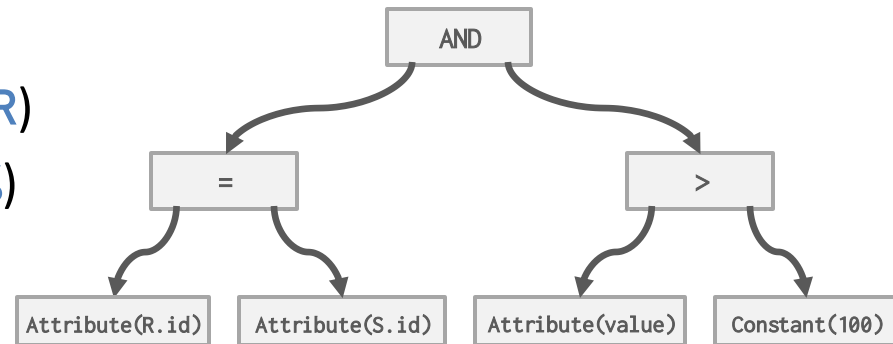
Vectorization Model

- Ideal for OLAP queries because it greatly reduces the number of invocations per operator.
- Allows a modern CPU to efficiently execute operators over batches of tuples.
 - Operators perform work in tight for-loops over arrays, which compilers know how to optimize / vectorize.
 - No data or control dependencies.
 - Hot instruction cache.

Expression Evaluation

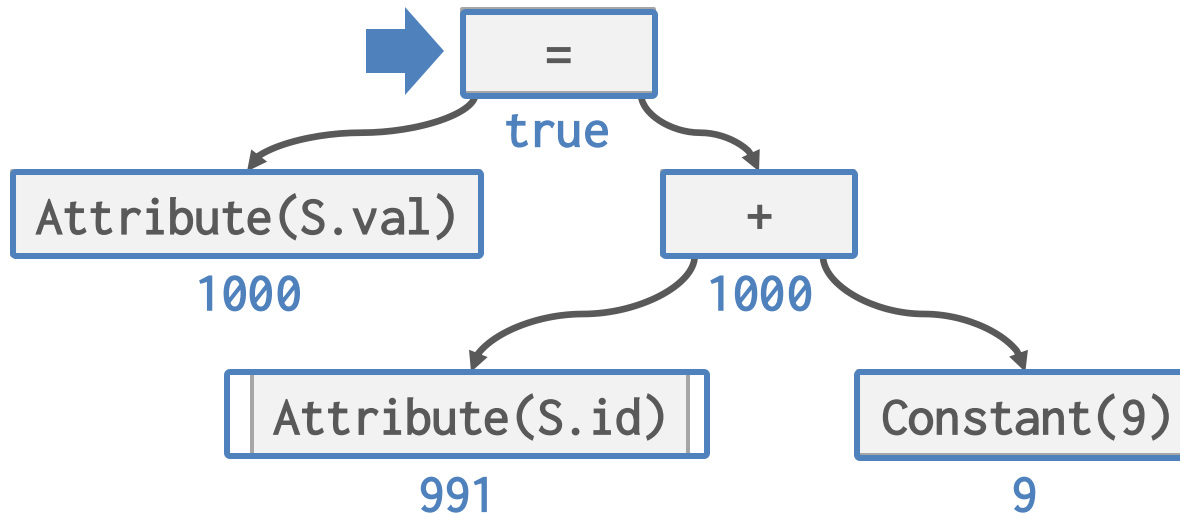
- The DBMS represents snippets of compute (e.g., **WHERE** clause) as expression trees.
- The nodes in the tree represent different expression types:
 - Comparisons (**=**, **<**, **>**, **!=**)
 - Conjunction (**AND**), Disjunction (**OR**)
 - Arithmetic Operators (**+**, **-**, *****, **/**, **%**)
 - Constant Values
 - Tuple Attribute References
 - Functions

```
SELECT R.id, S.cdate  
FROM R JOIN S  
ON R.id = S.id  
WHERE S.value > 100;
```



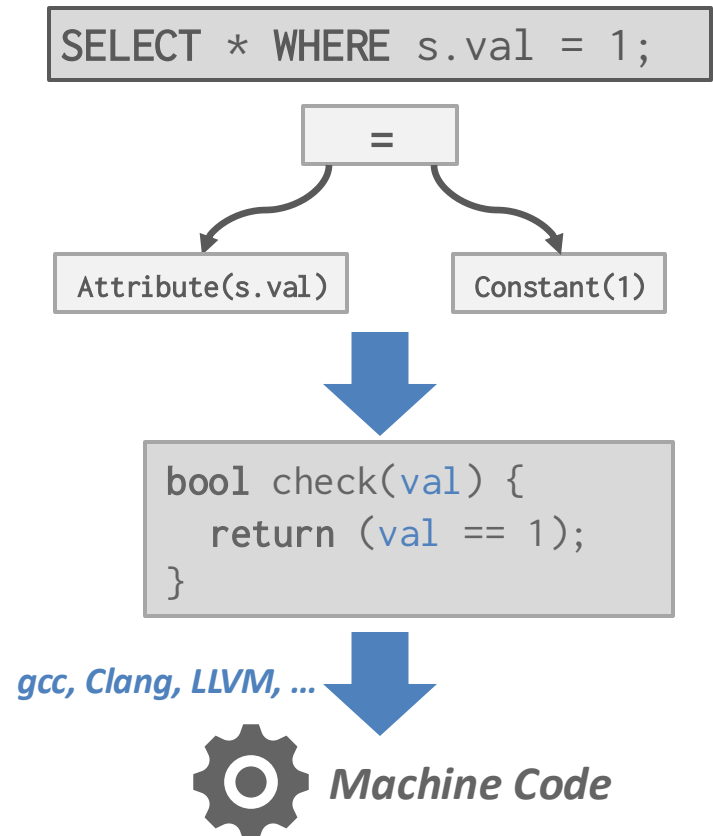
Expression Evaluation

```
SELECT * FROM S  
WHERE S.val = S.id + 9
```



Expression Evaluation

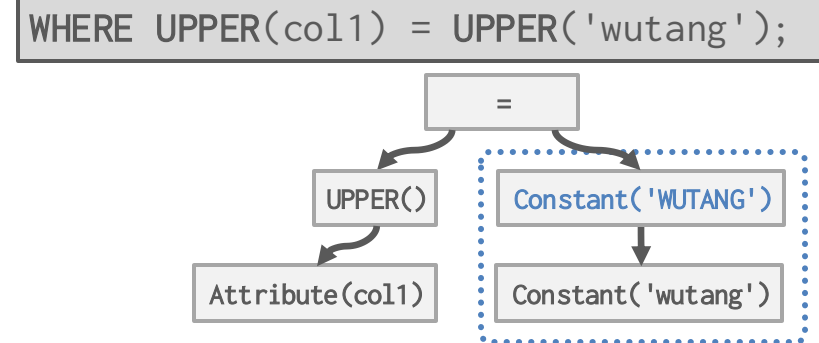
- Evaluating predicates by traversing a tree is terrible for the CPU.
 - The DBMS traverses the tree and for each node that it visits, it must figure out what the operator needs to do.
- A better approach is to compile and evaluate the expression directly
 - Bonus: vectorized execution



Optimizations

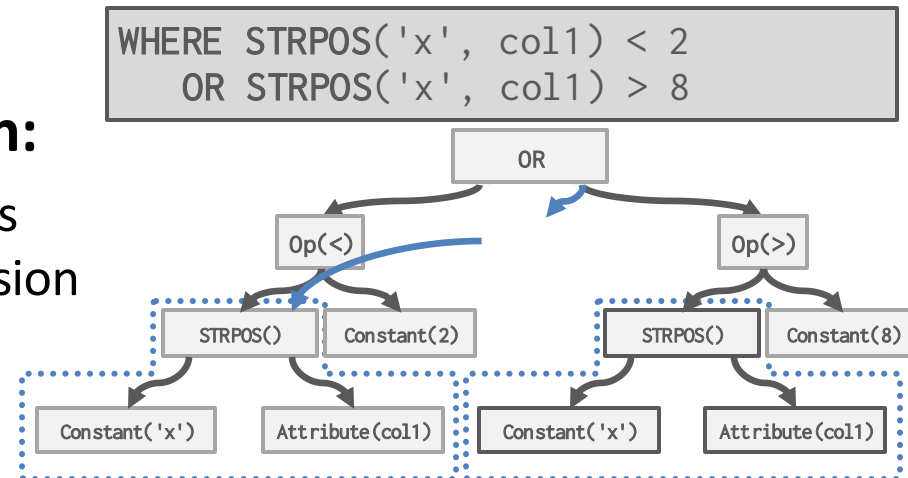
- **Constant Folding:**

- Identify redundant / unnecessary operations that are wasteful.
- Compute a sub-expression on a constant value once and reuse result per tuple.



- **Common Sub-Expr. Elimination:**

- Identify repeated sub-expressions that can be shared across expression tree.
- Compute once and then reuse result.



- **Many More!**

Query Compilation

- Idea: All of classical DBMS executor is essentially an interpreted engine, not just expressions
- Some modern systems compile queries to machine code directly
 - Data-drive execution (i.e. push-based) allows us to write operator trees as flat for-loops over data
 - Compilers are really good at optimizing this
 - Can apply Just-in-Time (JIT) compilation techniques

Vectorization vs. Compilation

- Compiled query engines are extremely complex and hard to maintain
 - And a nightmare to debug
- Most DB practitioners agree that a well-optimized vectorized execution achieves most of the speed up from query compilation with a fraction of the engineering cost

Fun Fact

- Query Compilation was in the first RDBMS (IBM System R) from 1970s!
 - Handwritten assembly templates
- Technique abandoned in commercial product
 - Too much work
 - Poor portability

COMPUTING PRACTICES

A History and Evaluation of System R

Donald D. Chamberlin	Thomas G. Price
Morton M. Astrahan	Franco Putzolu
Michael W. Blasgen	Patricia Griffiths Selinger
James N. Gray	Mario Schkolnick
W. Frank King	Donald R. Slutz
Bruce G. Lindsay	Irving L. Traiger
Raymond Lorie	Bradford W. Wade
James W. Mehl	Robert A. Yost

IBM Research Laboratory
San Jose, California

1. Introduction

Throughout the history of information storage in computers, one of the most readily observable trends has been the focus on data independence. C.J. Date [27] defined data independence as "immunity of applications to change in storage structure and access strategy." Modern database systems offer data independence by providing a high-level user interface through which users deal with the information content of their data, rather than the various bits, pointers, arrays, lists, etc. which are used to represent that information. The system assumes responsibility

SUMMARY: System R, an experimental database system, was constructed to demonstrate that the usability advantages of the relational data model can be realized in a system with the complete function and high performance required for everyday production use. This paper describes the three principal phases of the System R project and discusses some of the lessons learned from System R about the design of relational systems and database systems in general.

representation for the information; indeed, the representation of a given fact may change over time without

sent by connections between the relevant part and supplier records. In such a system, a user frames a mes-

Summary

- Database query engines are often organized as a tree of iterators
- Iterators are composable and easy to reason about, but has performance problems
- Many techniques to sidestep those problems in modern systems

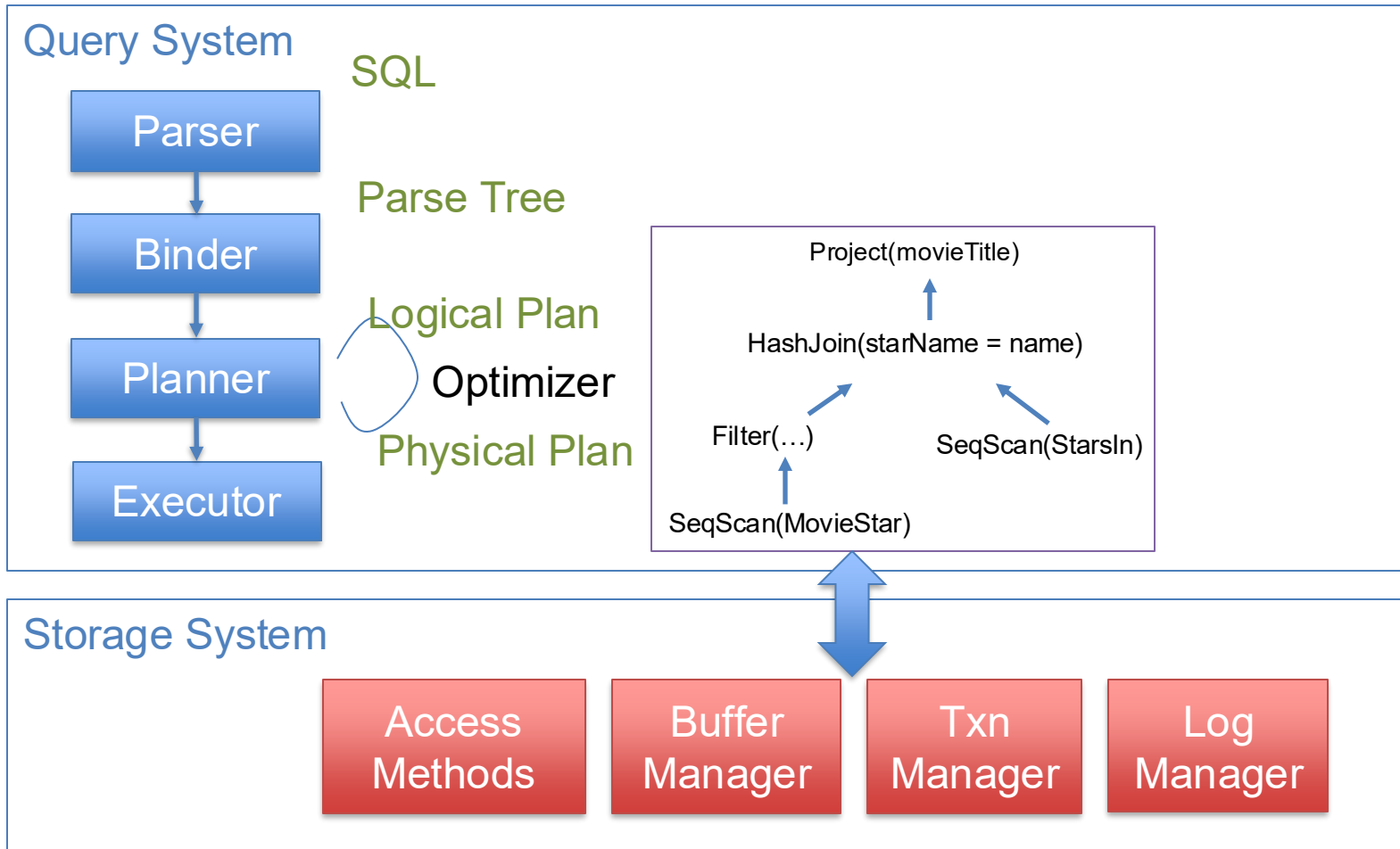
Let's Take a Short Break

- We will come back to walk through a few examples from GoDB

GoDB

- Hopefully you are finishing up Lab 1!
 - We will have in-class write-up on 3/5
- Lab 2 Releases Today!
 - Due 3/19
- Rest of Today: GoDB Architecture Overview

GoDB



GoDB Frontend

- Command Line Tool similar to SQLite
 - Limited in features
- Schema must be statically loaded through json
 - Data may be bootstrapped with csv
 - Data is persistent
- Once data is loaded, can interact with database using standard SQL
 - Most basic commands are supported

GoDB Frontend

- Simple Rule-based Optimizer Supplied
 - As you implement operators, you can enable them in `main.go`
- You are not required to modify it
 - Although you should understand what it does
- More on optimizers in lec 8

GoDB

- Executors vs. PlanNodes
 - Plan nodes are immutable objects for the optimizer, holding immutable information (e.g., expressions)
 - Executors hold execution-time resources and physical pointers
 - Separation is necessary for performance and to facilitate plan caching

GoDB Expressions

- Interpreted version provided for you
- Beware the type system and NULLs!

GoDB Execution Engine

- Normally, DBMS operators must work with larger-than-memory data
 - Operators internally must manage spilling to disk
 - Implementations often must be optimized for disk
- For simplicity, in GoDB, you may assume intermediate state fits in memory

GoDB Indexes

- GoDB support hash and B-tree indexes
 - For simplicity, these indexes are in-memory only
 - You may assume indexes are thread-safe
 - Indexes are simply rebuilt when loading the database
- You are responsible for writing index lookup/scans and maintaining them during modification

Choose Your Own Adventure

- In Lab 2, you will implement all the basic executors to support basic data processing
 - Includes access methods, aggregation, sorting, join, etc.
- You can then pick from a range of options to build advanced operators that speed up some operations
 - Pick 2 out of 5

Next Class

- Join Algorithms
- We will circle back to Normalization in lecture 9
- Good Luck on Lab 2!